

Bàn về một số phương pháp và kỹ thuật tối ưu tăng thấp của chương trình

Luo Keqiang
Trường Trung học số 7 Thành Đô

Luận văn Trại đông 2009

Nguồn gốc: Bàn về một số phương pháp và kỹ thuật tối ưu tăng thấp của chương trình

IOI China candidate team papers / 2009 / Luo Keqiang

Abstract

Bài viết lấy mục tiêu tối ưu hiệu quả thời gian của chương trình, nhìn từ các khái niệm tương đối tăng thấp như trình biên dịch, mã hợp ngữ và đặc tính của CPU. Nội dung thảo luận toàn diện nhiều mặt của tối ưu chương trình, tổng kết các tư tưởng, nguyên tắc, phương pháp và mẹo thực dụng trong tối ưu, đồng thời thử đánh giá giá trị áp dụng của chúng trong thi Tin học.

Từ khóa. Tối ưu; CPU; hợp ngữ; trình biên dịch.

Contents

1	Lời nói đầu	1
1.1	Thuật toán là yếu tố quyết định	1
1.2	Giới hạn của độ phức tạp thời gian	1
1.3	Thuật toán không phải toàn bộ câu chuyện	2
1.4	Vì sao cần học tối ưu tăng thấp	2
1.5	Nền tảng thử nghiệm	2
2	Ví dụ dẫn nhập	3
2.1	Cài đặt mặc mặc	3
2.2	Tối ưu lần thứ nhất	3
2.3	Tối ưu lần thứ hai	4
2.4	Tối ưu lần thứ ba: hợp ngữ nội tuyến	4
2.5	Tối ưu lần thứ tư: giảm phụ thuộc	5
2.6	Tối ưu lần thứ năm: SIMD	6
2.7	Tối ưu lần thứ sáu: SSE4	7
3	Hiệu quả chạy của các lệnh CPU	8

4	Tối ưu phép toán số học	8
4.1	Phép chia	8
4.1.1	Loại bỏ phép chia	8
4.1.2	Giảm chia số thực	9
4.1.3	Các phép chia đặc biệt	9
4.2	Phép nhân	11
4.2.1	Dùnglea cho nhân hằng số	11
4.2.2	Tối ưu mul_mod	11
4.3	Số nguyên lớn	12
5	Đặc tính tối ưu của CPU	12
5.1	Cache	12
5.1.1	Truy cập bộ nhớ chậm	12
5.1.2	Thí nghiệm quan sát cache	12
5.1.3	Viết mã thân thiện với cache	13
5.1.4	Cache mã lệnh	14
5.2	Dự đoán nhánh	14
5.2.1	Loại bỏ nhánh	15
5.3	Thực thi ngoài thứ tự	15
5.3.1	Giảm phụ thuộc giữa câu lệnh	16
6	Kỹ thuật bit	16
6.1	Nén bit	16
6.2	Thống kê trong một gói bit	16
6.3	Loại bỏ nhánh bằng C	17
7	Lưu ý khi dùng mảng nhiều chiều	18
7.1	Chi phí định địa chỉ	18
7.2	Hiệu quả tăng thấp của truy cập mảng	19
8	Ví dụ ứng dụng	19
8.1	Số Mersenne	19
8.2	Vũ điệu lộn lẫy	19
8.3	Dịch tác phẩm Maya	20
9	Tổng kết	20
10	Tài liệu tham khảo	20
11	Lời cảm ơn	21

1 Lời nói đầu

Olympiad in Informatics là cuộc thi về cách viết chương trình máy tính để giải một bài toán cụ thể. Điểm then chốt là dùng tài nguyên hệ thống hữu hạn, gồm thời gian CPU và bộ nhớ, để giải các mô hình toán học có quy mô lớn. Khi đã đặt tính đúng làm tiền đề, “hiệu quả” tự nhiên trở thành yếu tố đầu tiên cần xét. Hiệu quả có hai mặt: thời gian và không gian. Bài viết này tập trung vào tối ưu hiệu quả thời gian.

1.1 Thuật toán là yếu tố quyết định

Nói ngắn gọn, tối ưu thời gian là dùng mọi biện pháp có thể để chương trình chạy nhanh hơn trong khi vẫn đúng. Công cụ quan trọng nhất vẫn là thuật toán tốt. Thuật toán là một dãy bước cơ học với ngữ nghĩa xác định, mô tả phương pháp chung để giải một mô hình toán học. Thuật toán tốt hay xấu trực tiếp quyết định hiệu quả chạy; khác biệt giữa thuật toán kém và thuật toán tốt giống như khác biệt giữa đi bộ và đi máy bay, hoàn toàn không cùng bậc.

1.2 Giới hạn của độ phức tạp thời gian

Để đo hiệu quả thời gian của thuật toán, ta thường dùng ký hiệu big-O. Khi nói thời gian chạy bị chặn bởi $\mathcal{O}(f(n))$, nghĩa là nếu thời gian thực tế theo kích thước dữ liệu n là $T(n)$, thì tồn tại các hằng số dương n_0, c sao cho với $n \geq n_0$,

$$0 \leq T(n) \leq cf(n).$$

Cận này giúp ta biết tốc độ tăng của $T(n)$, từ đó ước lượng thuật toán có thể chạy trong thời gian hợp lý hay không.

Nhưng công cụ đó rất thô. Hằng số c trong định nghĩa không quan trọng về mặt tiệm cận, nhưng trong thực tế lại không thể bỏ qua. Cùng là thuật toán $\mathcal{O}(n)$, có chương trình chỉ tốn $2n$ chu kỳ CPU, có chương trình tốn hơn $1000n$ chu kỳ. Vì vậy hai cài đặt cùng độ phức tạp vẫn có thể khác nhau rất xa về thời gian thực.

1.3 Thuật toán không phải toàn bộ câu chuyện

Hằng số trong thời gian chạy chịu ảnh hưởng của nhiều yếu tố. Một phần thuộc tầng logic: thuật toán, cách biểu diễn, cách đơn giản hóa điều kiện. Nhưng còn một phần nằm dưới tầng logic, khó nhìn thấy hơn nhưng vẫn ảnh hưởng rõ rệt. Ví dụ, khi viết $a/=7$ trong C, bộ xử lý có thể không dùng phép chia chậm, mà thay bằng phép nhân và dịch bit.

Vì vậy có thể nhìn hằng số c trong big-O như $c = c_1 c_2$. c_1 là chi phí tầng logic, còn c_2 là chi phí tầng thấp của từng câu lệnh. Thời gian chạy xấp xỉ

$$c_2 \cdot [c_1 f(n)].$$

Phần trong ngoặc phụ thuộc vào thuật toán; c_2 phụ thuộc vào cách chương trình được triển khai ở tầng dưới. Bài viết này nghiên cứu cách tối ưu những chi tiết đó.

1.4 Vì sao cần học tối ưu tầng thấp

Trong OI, thuật toán là trọng tâm. Tối ưu tầng thấp có vẻ không quan trọng và không có không gian cải thiện lớn. Tuy vậy khi thuật toán của ta có khiếm khuyết bẩm sinh, tinh chỉnh tầng thấp có thể giúp đáng kể; phần sau sẽ nêu vài ví dụ thực tế.

Ngoài thi đấu, hiểu tầng thấp giúp ta hiểu sâu hơn cỗ máy đang dùng. Ngay cả khi viết bằng ngôn ngữ bậc cao, ta vẫn hình dung được việc gì đang xảy ra bên dưới, từ đó viết mã có chất lượng tốt hơn.

1.5 Nền tảng thử nghiệm

Do điều kiện hạn chế, phần nghiên cứu, lập trình và đo đạc chính được thực hiện trên một máy ThinkPad X61. Cấu hình liên quan tới tối ưu:

CPU	Intel Core 2 Duo T7100 @ 1.80GHz
Cache	L1 64KB, L2 2MB
Bộ nhớ chính	SODIMM synchronous 667 MHz, 1GB × 2
Hệ điều hành	Ubuntu 8.10 Intrepid Ibex
Nhân	Linux 2.6.27-8-generic
Trình biên dịch	gcc 4.3.2
Assembler	GNU assembler 2.18.93.20081009
Linker	GNU ld 2.18.93.20081009

Tác dụng của một kỹ thuật tối ưu phụ thuộc mạnh vào nền tảng, đặc biệt là CPU. Vì vậy tác giả cũng thử trên nhiều hệ điều hành và nhiều CPU của Intel, AMD. Giá trị tuyệt đối khác nhau rất nhiều, nhưng hiệu quả tương đối của các hướng tối ưu cơ bản khá nhất quán, vì CPU hiện đại dùng kiến trúc nội bộ và cơ chế tối ưu tương tự nhau. Điều cần nắm là nguyên tắc lớn, không phải các mẹo kỳ dị chỉ hợp một CPU. Riêng nhóm tối ưu SIMD được nhắc ở sau tuy tương thích trên chip AMD nhưng hiệu quả không nổi bật bằng.

2 Ví dụ dẫn nhập

Xét một nhiệm vụ rất đơn giản: viết hàm nhận con trỏ tới đầu mảng số nguyên dương 32 bit và độ dài mảng, rồi trả về giá trị lớn nhất. Phần đầu hàm trong C là:

```
int get_max(int* a,int l);
```

2.1 Cài đặt mộc mạc

Ở tầng thuật toán không có gì đáng bàn: quét tuyến tính là đủ, và vì mọi phần tử đều phải được xét để bảo đảm đúng, cận dưới cũng là $\mathcal{O}(n)$.

```
// phiên bản đầu tiên
int get_max(int* a,int l){
    int mx=0,i;
    for(i=0;i<l;i++){
        if(a[i]>mx)mx=a[i];
    }
    return mx;
}
```

Các chương trình trong bài được biên dịch theo cách gần môi trường thi, không bật tùy chọn tối ưu:

```
gcc a.c -o a
```

Trên mảng 10 000 000 số nguyên, chạy liên tiếp 100 lần rồi lấy trung bình, hàm trên tốn 75 305 510 chu kỳ. Mỗi số mất khoảng 7 đến 8 chu kỳ, kết quả chưa thỏa đáng.

Hàm đo thời gian dùng lệnh `rdtsc` của IA-32:

```
#define ull unsigned long long
ull get_clock(){
    ull ret;
    __asm__ __volatile__ ("rdtsc\n\t":"=A"(ret):);
    return ret;
}
```

Phương pháp này khá chính xác nhưng không loại được chu kỳ do hệ điều hành và tiến trình nền chiếm dụng; có thể kiểm tra chéo bằng lệnh `time`.

2.2 Tối ưu lần thứ nhất

Trong mã đầu, biểu thức `a[i]` xuất hiện hai lần. Chi phí tính lại địa chỉ trong vòng lặp nhỏ không phải bằng không, nên ta đổi sang duyệt con trỏ:

```
// toi uu lan 1
int get_max(int* a,int l){
    int mx=0,*ed=a+l;
    while(a!=ed){
        if(*a>mx)mx=*a;
        a++;
    }
    return mx;
}
```

Thời gian trung bình giảm còn 66 047 005 chu kỳ, tức mỗi phần tử giảm khoảng một chu kỳ, nhanh hơn bản đầu khoảng 12%. Đây vẫn là tối ưu ở tầng C và có thể giải thích bằng cách giảm lặp lại phép định địa chỉ.

2.3 Tối ưu lần thứ hai

Nếu tiếp tục cùng hướng, ta khó nghĩ ra cải thiện lớn. Điểm mới là tách tiến trình tìm max thành tám luồng độc lập rồi gộp kết quả:

```
// toi uu lan 2
int get_max(int* a,int l){
    assert(l%8==0);
    #define D(x) mx##x=0
    int D(0),D(1),D(2),D(3),
        D(4),D(5),D(6),D(7),*ed=a+l;
    #define CMP(x) if(*(a+x)>mx##x)mx##x=*(a+x);
    while(a!=ed){
        CMP(0);CMP(1);
        CMP(2);CMP(3);
        CMP(4);CMP(5);
        CMP(6);CMP(7);
        a+=8;
    }
    #define CC(x1,x2) if(mx##x1>mx##x2)mx##x2=mx##x1;
```

```

    CC(1,0);CC(3,2);
    CC(5,4);CC(7,6);
    CC(2,0);CC(6,4);
    CC(4,0);
    return mx0;
}

```

Mã dùng nhiều macro và chỉ xử lý trực tiếp trường hợp l chia hết cho 8, nhưng ý tưởng rất đơn giản. Kết quả đo là 34 818 706 chu kỳ, nhanh hơn bản đầu 54%. Nếu chỉ nhìn tầng C, khó giải thích mức cải thiện này: số lần tăng con trỏ giảm xuống 1/8, nhưng thao tác đó nhỏ hơn nhiều so với phần còn lại của vòng lặp.

Điểm mấu chốt là mã sau khi biên dịch. Ở bản đầu, mỗi lần cập nhật mx phụ thuộc vào kết quả lần trước, nên các lệnh liên quan buộc phải đi nối tiếp. Khi tách thành $mx0, mx1, \dots$, các chuỗi lệnh không còn phụ thuộc nhau, CPU có thêm cơ hội chạy song song bằng cơ chế nội bộ như pipeline, dự đoán nhánh, đổi tên thanh ghi, cache và thực thi ngoài thứ tự. Chi tiết sẽ được giải thích ở các phần sau.

2.4 Tối ưu lần thứ ba: hợp ngữ nội tuyến

Khi đã nhìn tới tầng hợp ngữ, ta có thể viết trực tiếp đoạn nóng bằng hợp ngữ. Trong CPU, các khái niệm như thanh ghi, cờ trạng thái và lệnh là trung tâm, nhưng bị ngôn ngữ bậc cao che đi. C cũng dựa nhiều vào biến trong bộ nhớ, mà đọc ghi bộ nhớ là một trong các thao tác chậm. Hợp ngữ cho phép thao tác trực tiếp hơn và dùng các lệnh đặc biệt. Đổi lại, chi phí lập trình rất cao, khó bảo trì và phụ thuộc nền tảng. Cách thực dụng trong thi là dùng hợp ngữ nội tuyến cho đoạn nghẽn cổ chai, còn phần chính vẫn viết bằng C.

Các đoạn sau dùng hợp ngữ nội tuyến của gcc, cú pháp AT&T:

```

// toi uu lan 3
int get_max(int* a,int l){
    int ret;
    __asm__ __volatile__ (
        "movl    $0, %%eax\n\t"
        ".p2align 4,,15\n\t"
        "LP1:\n\t"
        "cmpl    -4(%1,%2,4), %%eax\n\t"
        "jge ED\n\t"
        "movl    -4(%1,%2,4), %%eax\n\t"
        "ED:\n\t"
        //"loop LP1\n\t"
        "decl    %2\n\t"
        "jnz LP1\n\t"
        "movl    %%eax, %0\n\t"
        : "=m"(ret)
        : "r"(a), "c"(l)
        : "%eax");
    return ret;
}

```

Thời gian trung bình là 21 322 853 chu kỳ, chỉ khoảng 2 chu kỳ cho một phần tử, nhanh hơn bản đầu 72%. Vòng lặp lỗi vẫn có 5 lệnh, trong đó có hai nhánh điều kiện và hai lần truy cập bộ nhớ bằng định

địa chỉ SIB phức tạp. Kết quả tốt chủ yếu nhờ cache dữ liệu, dự đoán nhánh và thực thi ngoài thứ tự.

Risc và Cisc. Trong mã trên có một lệnh loop bị chú thích. Trực giác cũ có thể nghĩ loop ngắn hơn và phức hợp hơn nên nhanh hơn. Thực tế thay `decl/jnz` bằng loop làm thời gian tăng lên 56 457 348 chu kỳ.

Lý do nằm ở lịch sử Cisc và Risc. x86 là kiến trúc Cisc với tập lệnh lớn và nhiều lệnh phức tạp. Trên CPU hiện đại, nhiều lệnh x86 được dịch bên trong thành các vi lệnh giống Risc. Các nhà sản xuất tối ưu rất mạnh cho nhóm lệnh đơn giản phù hợp pipeline, trong khi một số lệnh phức tạp cũ như loop không còn có ưu thế. Vì vậy khi tối ưu hợp ngữ hiện nay, thường nên dùng tập con lệnh cơ bản đã được tối ưu tốt. Ngoại lệ quan trọng là các tập lệnh SIMD được đầu tư riêng.

2.5 Tối ưu lần thứ tư: giảm phụ thuộc

Ta thử lặp lại ý tưởng tách chuỗi phụ thuộc trong hợp ngữ:

```
// tối ưu lần 4
int get_max(int* a, int l){
    assert(l%2==0);
    int ret;
    __asm__ __volatile__ (
        "movl    $0, %%eax\n\t"
        "movl    $0, %%edx\n\t"
        ".p2align 4,,15\n"
        "LP2:\n\t"
        "cmpl    (%1),    %%eax\n\t"
        "jge ED2\n\t"
        "movl    (%1),    %%eax\n\t"
        "ED2:\n\t"
        "cmpl    4(%1), %%edx\n\t"
        "jge ED3\n\t"
        "movl    4(%1), %%edx\n\t"
        "ED3:\n\t"
        "addl    $8, %1\n\t"
        "subl    $2, %2\n\t"
        "jnz LP2\n\t"
        "cmpl    %%edx, %%eax\n\t"
        "cmovll %%edx, %%eax\n\t"
        "movl    %%eax, %0\n\t"
        : "=m"(ret)
        : "r"(a), "r"(l)
        : "%eax", "%edx");
    return ret;
}
```

Kết quả là 17 447 544 chu kỳ, có cải thiện nhưng không thần kỳ như bản C tám luồng. Khi thử mở rộng bốn luồng và dùng hết thanh ghi tổng quát, tốc độ lại giảm. Nguyên nhân là bản hợp ngữ đã rất gọn; khi lệnh tải đầu vòng lặp bị cache miss, phía sau không còn nhiều lệnh độc lập để CPU kéo lên chạy trước. Thêm quá nhiều nhánh và dùng hết thanh ghi còn gây bất lợi cho RAT và dự đoán nhánh. Tối ưu không thể dựa vào tưởng tượng.

2.6 Tối ưu lần thứ năm: SIMD

SIMD là “single instruction, multiple data”: một lệnh xử lý đồng thời nhiều phần tử dữ liệu. Nó khác với lệnh phức hợp kiểu “một lệnh làm nhiều việc”; ở đây cùng một thao tác được áp vào nhiều phần tử đã được đóng gói trong thanh ghi rộng. Intel bắt đầu từ MMX, rồi SSE với thanh ghi 128 bit, sau đó SSE2, SSE3, SSE4; AMD cũng tương thích rộng rãi.

```
// toi uu lan 5
int get_max(int* a,int l){
    assert(l%4==0);
    assert(sse2);
    int ret,tmp[4];
    __asm__ __volatile__ (
        "\txorps %%xmm0,          %%xmm0\n"
        "LP3:\n"
        "\tmovdqa          %%xmm0,          %%xmm1\n"
        "\tpcmptd          (%1),          %%xmm1\n"
        "\tandps %%xmm1,          %%xmm0\n"
        "\tandnps (%1),          %%xmm1\n"
        "\torps          %%xmm1,          %%xmm0\n"
        "\taddl $16, %1\n"
        "\tsubl $4, %2\n"
        "\tjnz          LP3\n"
        "\tmovdqu          %%xmm0,          (%3)\n"
        "\tmovl (%3),          %%eax\n"
        "\tcmpl 4(%3), %%eax\n"
        "\tcmovll 4(%3), %%eax\n"
        "\tcmpl 8(%3), %%eax\n"
        "\tcmovll 8(%3), %%eax\n"
        "\tcmpl 12(%3), %%eax\n"
        "\tcmovll 12(%3), %%eax\n"
        "\tmovl %%eax, %0\n"
        : "=m"(ret)
        : "r"(a), "r"(l), "r"(tmp)
        : "%eax");
    return ret;
}
```

Bốn giá trị lớn nhất cục bộ được đóng gói trong xmm0. Để lấy max giữa bốn số liên tiếp trong bộ nhớ và bốn số trong xmm0, mã dùng so sánh SSE tạo mặt nạ rồi ghép bằng phép bit. Dù cần tới 5 lệnh SSE, thời gian giảm còn 15 898 751 chu kỳ. Bản này chỉ tốn khoảng 1/5 thời gian bản đầu, tức xấp xỉ 1.5 chu kỳ cho mỗi phần tử.

2.7 Tối ưu lần thứ sáu: SSE4

Trên các CPU Penryn 45nm của Intel, SSE4 có lệnh pmaxsd, có thể thay trực tiếp khối so sánh và phép bit:

```
// toi uu lan 6
```



```

int get_max(int* a,int l){
    assert(l%4==0);
    assert(sse4);
    int ret,tmp[4];
    __asm__ __volatile__ (
        "\txorps %%xmm0,          %%xmm0\n"
        "LP4:\n"
        "\tptestd      (%1),  %%xmm0\n"
        "\taddl $16, %1\n"
        "\tsubl $4, %2\n"
        "\tjnz      LP4\n"
        "\tmovdqu      %%xmm0,      (%3)\n"
        "\tmovl (%3),          %%eax\n"
        "\tcmpl 4(%3), %%eax\n"
        "\tcmovll 4(%3), %%eax\n"
        "\tcmpl 8(%3), %%eax\n"
        "\tcmovll 8(%3), %%eax\n"
        "\tcmpl 12(%3), %%eax\n"
        "\tcmovll 12(%3), %%eax\n"
        "\tmovl %%eax, %0\n"
        : "=m"(ret)
        : "r"(a), "r"(l), "r"(tmp)
        : "%eax");
    return ret;
}

```

Máy thử nghiệm của tác giả chưa chạy được SSE4 nên không có số đo, nhưng tác giả ước đoán nó có thể tiến gần mức cực hạn một chu kỳ cho mỗi phần tử.

3 Hiệu quả chạy của các lệnh CPU

Trên kiến trúc CISC, rất khó nói một lệnh cụ thể luôn tốn bao nhiêu chu kỳ: vi lệnh của nó có thể bị tách ra và chạy xen với lệnh khác; dữ liệu có ở cache hay không, có căn chỉnh hay không cũng ảnh hưởng lớn. Tuy vậy vẫn cần biết đại khái lệnh nào nhanh chậm. Một lệnh chia gần như luôn chậm hơn cộng.

Tác giả đo bằng cách đặt lệnh cần kiểm tra trong một ngữ cảnh có ý nghĩa, chèn `cpuid` trước và sau để cô lập, lặp 1 000 000 lần, rồi trừ chi phí vòng lặp không có lệnh đó. Kết quả trên máy thử nghiệm:

Lệnh	Toán hạng	Chu kỳ trung bình
<code>add</code>	<code>r32/r32</code>	1
<code>shr</code>	<code>imm8/r32</code>	1
<code>bsr</code>	<code>r32/r32</code>	1
<code>mul</code>	<code>r32</code>	2
<code>div</code>	<code>r32</code>	21
<code>fadd</code>	<code>r</code>	3
<code>fdiv</code>	<code>r</code>	35
<code>fsqrt</code>	<code>n/a</code>	60
<code>padd</code>	<code>xmm/xmm</code>	1

4 Tối ưu phép toán số học

4.1 Phép chia

Trên CPU hiện đại, phép nhân đã được tối ưu rất nhanh, nhưng phép chia vẫn phức tạp và tốn hàng chục chu kỳ. Nếu 20 lệnh dịch bit có thể thay một lệnh chia, việc đó thường đáng làm. Tuy nhiên không tồn tại thuật toán chia tổng quát nhanh hơn phần cứng CPU; nếu có, nhà sản xuất đã đưa nó vào mạch chia. Không gian tối ưu chỉ xuất hiện trong các trường hợp đặc biệt.

4.1.1 Loại bỏ phép chia

Cách hiệu quả nhất là tránh chia. Ví dụ điều kiện $\text{if}(a/b < c)$ đôi khi có thể đổi thành $\text{if}(a < b * c)$, nhưng phải xét tràn số và dấu của b . Trong thiết kế thuật toán cũng có những biến đổi tương tự, nhưng cần rất cẩn thận.

Trong các bài đếm modulo M , lạm dụng $\%M$ có thể rất đắt vì phép lấy dư ở tầng dưới dùng cùng lệnh với phép chia. Chẳng hạn để chuẩn hóa một số có thể âm, cách thường viết $(a \% M + M) \% M$ dùng hai phép chia. Có thể viết:

```
inline int mod(int a){a%=M;if(a<0)a+=M;return a;};
```

Trong nhân ma trận cho truy hồi tuyến tính, nếu lấy modulo ở mọi bước trong vòng lặp trong cùng, chương trình sẽ chậm đáng kể. Thường có thể cộng dồn tạm thời trong phạm vi an toàn rồi mới lấy modulo sau khi tính xong một phần tử.

4.1.2 Giảm chia số thực

Với số thực, biến đổi linh hoạt hơn vì quy luật đại số quen thuộc hơn. Nếu cần chia nhiều lần cho b , nên tính trước $1/b$ rồi nhân. Với

$$a/b + c/d,$$

có thể đổi thành

$$(ad + cb)/(bd),$$

tăng 3 phép nhân nhưng giảm 1 phép chia; trên máy tác giả cách này nhanh hơn hơn hai lần khi lặp 10 000 000 lần.

Ngay cả ba phép chia độc lập:

```
x=a/b;
```

```
y=c/d;
```

```
z=e/f;
```

có thể đổi thành:

```
t=1/(b*d*f);
```

```
x=a*d*f*t;
```

```
y=c*b*f*t;
```

```
z=e*b*d*t;
```

Mã thứ hai chỉ tốn khoảng 1/3 thời gian mã đầu trong thử nghiệm. Dĩ nhiên không nên lạm dụng: quá nhiều lệnh đơn giản thay một lệnh phức tạp có thể làm mã khó đọc, khó kiểm chứng và ảnh hưởng độ chính xác.

4.1.3 Các phép chia đặc biệt

Nếu toán hạng chắc chắn không âm, nên chuyển sang kiểu không dấu trước khi chia:

```
a=(unsigned int)b/123;
```

Chia không dấu thường nhanh hơn chia có dấu.

Chia cho lũy thừa của 2. Chia cho 2^k tương đương dịch phải k bit. Nếu modulo cũng là 2^k , có thể để số tràn tự nhiên và chỉ giữ k bit thấp.

Đôi khi cần tìm k khi biết $b = 2^k$. Một mẹo C dùng bảng băm không va chạm:

```
char tb[32]={31,0,27,1,28,18,23,2,
             29,21,19,12,24,9,14,3,
             30,26,17,22,20,11,8,13,
             25,16,10,7,15,6,5,4};
// x can be unsigned int
#define LOG(x) tb[((x)*263572066)>>27]
```

Ý tưởng là chọn một số T sao cho khi dịch trái T lần lượt 0 đến 31 bit, 5 bit cao chạy qua đủ các giá trị 0 đến 31.

Nhưng IA-32 đã có lệnh quét bit. `bsf` quét từ thấp lên cao, còn `bsr` từ cao xuống thấp:

```
inline unsigned int LOG2(unsigned int x){
    unsigned int ret;
    __asm__ __volatile__ ("bsrl %1, %%eax":"=a"(ret):"m"(x));
    return ret;
}
```

`LOG2` không chỉ xử lý lũy thừa của 2 mà trả về $\lfloor \log_2 x \rfloor$. Nó hữu ích cho bảng sparse table, LCA và nhiều thuật toán tìm kiếm theo bit. `bsf` hữu ích để đếm số bit 0 ở cuối, ví dụ trong Miller-Rabin hoặc GCD nhị phân.

Chia cho hằng số. Với số thực, chia cho hằng số chỉ cần nhân với nghịch đảo. Với số nguyên, không thể làm trực tiếp như vậy, nhưng trình biên dịch có kỹ thuật riêng. Khi biên dịch:

```
#include<stdio.h>
int main(){
    unsigned int a;
    scanf("%u",&a);
    a/=7;
    printf("%u",a);
    return 0;
}
```

gcc -O2 sinh đoạn tương đương:

```
movl    -8(%ebp), %ecx
movl    $613566757, %edx
movl    %ecx, %eax
```

```

mull %edx
subl %edx, %ecx
shrl %ecx
addl %ecx, %edx
shrl $2, %edx
movl    %edx, -8(%ebp)

```

Con số 613566757 thỏa $613566757 \cdot 7 = 2^{32} + 3$. Bằng cách nhân vào định nghĩa phép chia có dư $7a + q = c$, các phần “lộn xộn” chứa q nằm ở bit thấp và bị loại sau khi dịch phải đủ nhiều, còn thương xuất hiện ở bit cao. gcc dùng vài biến thể tùy hằng số. Dù gcc đã làm việc này ngay cả khi không bật tối ưu mạnh, hiểu nguyên lý cho thấy có thể mở rộng sang trường hợp mẫu số được biết muện nhưng dùng lặp lại, bằng cách tự sinh mã máy; tuy vậy cách đó quá đắt cho thi đấu.

Dùng trừ thay chia? Một số mã GCD cố thử trừ b khỏi a vài lần trước khi dùng phép chia. Tác giả thử nghiệm và thấy chậm hơn, kể cả khi viết hợp ngữ. Lý do là tốc độ lệnh chia không cố định; khi a và b gần nhau, nó có thể khá nhanh. Trong khi đó mô phỏng bằng trừ tạo nhiều nhánh điều kiện, làm CPU khó chạy trơn tru. Đây là ví dụ điển hình rằng tối ưu phải đo, không suy diễn.

4.2 Phép nhân

So với chia, phép nhân ít chỗ để nghiên cứu hơn. Trên CPU hiện đại, mul đã đủ nhanh; nhiều tối ưu cổ điển như viết $a \cdot 10$ thành $(a < 3) + (a < 1)$ có thể chậm hơn do cần hai dịch, một cộng và vài mov ẩn.

4.2.1 Dùng lea cho nhân hằng số

gcc nhân 10 có thể sinh:

```

lea (%eax,%eax,4),%eax
add %eax,%eax

```

lea offset(reg1,reg2,c),dest tính $reg1 + reg2 \cdot c + offset$, với $c \in \{1, 2, 4, 8\}$. Vì vậy dòng đầu nhân eax với 5, dòng sau nhân đôi thành 10. Một số ví dụ do gcc sinh:

```

nhan 2:  addl %eax,%eax
nhan 9:  leal (%eax,%eax,8),%eax
nhan 13: leal (%eax,%eax,2),%edx
         leal (%eax,%edx,4),%edx
nhan 17: movl %eax,%edx
         sall 4,%edx
         addl %edx,%eax
nhan 68: movl %eax,%edx
         sall 6,%edx
         leal (%edx,%eax,4),%eax

```

Không phải hằng số nào cũng có biểu diễn ngắn; với $a \cdot 14$, gcc dùng imul.

4.2.2 Tối ưu mul_mod

Một sự thật dễ bị che đi ở C là lệnh mul không “tràn” theo nghĩa mất kết quả: nhân hai số 32 bit cho kết quả 64 bit trong edx:eax. Lệnh div cũng lấy bị chia từ edx:eax. Vì vậy có thể viết:

```
#define M0 123456789
inline int mul_mod(int a,int b){
    int ret;
    __asm__ __volatile__ ("\tmull %%ebx\n\tdivl %%ecx\n"
        : "=d"(ret): "a"(a), "b"(b), "c"(M0));
    return ret;
}
```

Trên máy tác giả, hàm này nhanh hơn hơn hai lần so với:

```
inline int mul_mod_slow(int a,int b){
    return (long long)a*b%M0;
}
```

Khi modulo là số 64 bit, có một mẹo dùng số thực để xấp xỉ thương:

```
typedef long long ll;
#define MOL 123456789012345LL
inline ll mul_mod_ll(ll a,ll b){
    ll d=(ll)floor(a*(double)b/MOL+0.5);
    ll ret=a*b-d*MOL;
    if(ret<0)ret+=MOL;
    return ret;
}
```

Hai phép nhân trong $a*b - d*MOL$ có thể tràn, nhưng hiệu của chúng được dự đoán nằm trong 64 bit; phần tràn chỉ lệch 0 hoặc 1 lần modulo, rồi được sửa bằng kiểm tra dấu. Không nên dùng với modulo quá lớn vì sai số số thực.

4.3 Số nguyên lớn

Theo tác giả, mã số nguyên lớn vốn rất hợp với hợp ngữ. IA-32 có nhiều lệnh gần như được sinh ra cho số nguyên lớn; ở tầng hợp ngữ có thể lưu số theo cơ số 2^{32} để tận dụng lệnh và đạt tốc độ cao. Tác giả từng viết một thư viện nội tuyến nhanh hơn cách truyền thống vài lần.

5 Đặc tính tối ưu của CPU

CPU không chỉ đơn giản thực hiện từng lệnh theo thứ tự. Khi tần số xung bị giới hạn bởi công nghệ chế tạo và công suất, hiệu năng vẫn tăng nhờ các cơ chế như nạp trước lệnh, dự đoán nhánh, thực thi ngoài thứ tự và cache. Cùng một chức năng nhưng cách viết hơi khác có thể khiến CPU tối ưu tốt hoặc kém rất xa. Phần này chỉ giữ các điểm có ý nghĩa ở tầng ngôn ngữ bậc cao.

5.1 Cache

5.1.1 Truy cập bộ nhớ chậm

Biến trong ngôn ngữ bậc cao thường nằm trong bộ nhớ, còn tốc độ bộ nhớ kém xa tốc độ CPU. Khi cần dữ liệu, CPU phải gửi yêu cầu qua bus, đợi bộ nhớ chuẩn bị rồi lấy về, có thể tốn hàng chục hoặc hàng trăm chu kỳ. CPU giải quyết bằng hai hướng: trong lúc đợi thì làm việc khác, và dùng cache.

Cache dựa trên tính cục bộ: chương trình thường thực thi mã gần nhau và truy cập dữ liệu gần nhau; một vị trí vừa truy cập có xác suất cao sẽ được dùng lại sớm. Máy thử nghiệm có L1 64KB, L2 2MB; truy cập L1 gần như truy cập thanh ghi.

Khi tải một biến vào cache, CPU không tải từng byte riêng lẻ mà tải cả một dòng cache. Trên máy tác giả, mỗi dòng là 64 byte. L1 có 1024 dòng nhưng một địa chỉ chỉ có thể vào một nhóm nhất định; mỗi nhóm có 8 dòng. Nếu nhóm đầy, phải đẩy một dòng ra. Truy cập đã có trong cache là hit; chưa có là cache miss, rất đắt.

5.1.2 Thí nghiệm quan sát cache

```
int sum=0;
for(i=0;i<N;i++)sum+=a[i];
```

Với $N = 10\,000\,000$, biên dịch -O2, chạy mười lần:

Lần	Chu kỳ
0	167150043
1	24793497
2	27947781
3	25602750
4	27184617
5	18583821
6	15880428
7	15828174
8	15944328
9	16141635

Lần đầu chậm hơn khoảng một bậc vì mảng chưa được nạp vào cache.

5.1.3 Viết mã thân thiện với cache

Ưu tiên biến cục bộ. Dữ liệu trên stack được truy cập rất thường xuyên và tập trung; đỉnh stack thường nằm trong L1. Biến toàn cục cần địa chỉ 32 bit, làm mã lệnh dài hơn và tỷ lệ cache hit thường kém hơn.

Dùng struct cho dữ liệu liên quan. Hai mảng tách rời:

```
int x[MAXN],y[MAXN];
```

thường kém hơn:

```
struct point{int x,y};
point p[MAXN];
```

vì khi dùng x của một điểm, ta thường sắp dùng y của cùng điểm; đặt gần nhau giúp cùng vào một dòng cache.

Căn chỉnh dữ liệu. Nếu một struct point bị vắt qua ranh giới dòng cache, mỗi lần truy cập có thể phải tải hai dòng. gcc cho phép yêu cầu căn chỉnh:

```
int array[1024] __attribute__((aligned(64)));
```

Dùng union khi phù hợp. Nếu hai dữ liệu không bao giờ được dùng đồng thời, cho chúng chồng lên cùng vị trí bằng union vừa tiết kiệm bộ nhớ vừa tận dụng cache khi chuyển từ dữ liệu này sang dữ liệu kia.

Tránh cấp phát động dày đặc. Cấp phát và giải phóng động phải quản lý danh sách khối nhớ, các khối sinh ra có thể rời rạc và bất lợi cho cache. Với cấu trúc dữ liệu động trong thi, nên mở trước một pool đủ lớn rồi tự cấp phát node. Node nên là struct thay vì nhiều mảng rời.

Không phải mọi mảng đều nên đặt trên stack. Mảng rất lớn dễ tràn stack và làm con trỏ stack dao động mạnh, phá lợi thế cục bộ. Tính năng mảng cục bộ có kích thước runtime của gcc cũng không được khuyến nghị trong ngữ cảnh này.

5.1.4 Cache mã lệnh

Mã chương trình cũng nằm trong bộ nhớ. CPU có cache lệnh tương tự cache dữ liệu. Ở tầng C, hai nguyên tắc chính là đặt các hàm liên quan gần nhau và giữ vòng lặp nóng gọn.

Ví dụ:

```
for(int i=0;i<1000000;i++){
    proc1();
    proc2();
}
```

nếu proc1 và proc2 phức tạp và ít liên quan, có thể tách:

```
for(int i=0;i<1000000;i++)proc1();
for(int i=0;i<1000000;i++)proc2();
```

Dù lặp bộ đếm hai lần, mỗi vòng lặp nhỏ hơn, mã và dữ liệu liên quan có cơ hội ở cache tốt hơn; dự đoán nhánh cũng dễ hơn. Nguyên tắc tổng quát: thứ có liên quan nên đặt gần, thứ không liên quan nên tách ra.

5.2 Dự đoán nhánh

Điều kiện trong C được biên dịch thành các lệnh nhảy có điều kiện. Nhánh có điều kiện gây khó cho cache mã, trace cache và đặc biệt là pipeline. Một lệnh hiện đại đi qua nhiều bước: nạp trước, giải mã, đổi tên thanh ghi, thực thi, retire. Nếu tới một nhánh mà CPU không biết đường nào đúng, nó có thể dừng pipeline chờ kết quả, rất đắt. Do đó CPU dự đoán hướng nhánh và chạy trước. Nếu dự đoán sai, CPU phải quay lại và làm lại, có thể mất hàng chục chu kỳ.

Thí nghiệm:

```
int main(){
    for(int i=0;i<N;i++)a[i]=i%2;
    ull start=get_clock();
    for(int i=0;i<N;i++){
        if(a[i])a[i]=123123123;
        else a[i]=1234123;
    }
    printf("%llu\n",get_clock()-start);
    return 0;
}
```

Kết quả là 32 889 996 chu kỳ. Nếu đổi dòng khởi tạo thành:

```
for(int i=0;i<N;i++)a[i]=rand()%2;
```

kết quả tăng lên 102 583 557. Mã đo không đổi; khác biệt chỉ là mẫu dữ liệu. Chuỗi 0,1 xen kẽ có quy luật nên CPU học được, còn dữ liệu ngẫu nhiên thì không.

Dự đoán nhánh thường tốt khi nhánh luôn đi cùng một hướng, lặp một mẫu đơn giản, hoặc tương quan với một nhánh trước đó. Vì vậy nhánh do so sánh bộ đếm vòng lặp thường an toàn: hầu như luôn tiếp tục, chỉ sai ở lần thoát.

5.2.1 Loại bỏ nhánh

Một số ví dụ hợp ngữ:

```
// tri tuyet doi cua eax
cld
xorl %edx, %eax
subl %edx, %eax
```

Nếu eax không âm, edx=0; nếu âm, edx=-1, phép xor rồi trừ biến nó thành bù hai của số âm.

Gán ebx=eax nếu ebx>eax:

```
subl %ebx, %eax
sbbbl %edx, %edx
andl %eax, %edx
addl %edx, %ebx
```

Hàm so sánh dấu:

```
int cmp(int a){return (a>>31)+(-a>>31&1);}
```

Trong ngôn ngữ bậc cao, mỗi hạng tử trong biểu thức logic có thể sinh nhánh riêng, nên điều kiện trong if nên được viết gọn và rõ.

5.3 Thực thi ngoài thứ tự

Thực thi ngoài thứ tự (*out-of-order execution*) cố sắp lại các vi lệnh để tận dụng các đơn vị thực thi song song. Hai kỹ thuật nền tảng là vi lệnh và đổi tên thanh ghi.

Vi lệnh. Lệnh x86 đi vào CPU được giải mã thành các thao tác nhỏ kiểu RISC. Ví dụ:

```
addl (%esi), %eax
```

có thể tách thành một vi lệnh tải bộ nhớ và một vi lệnh cộng. Còn:

```
addl %eax, (%esi)
```

cần tải bộ nhớ, cộng, rồi ghi lại bộ nhớ. Khi đã tách thành vi lệnh, CPU có thể đưa chúng vào các cổng khác nhau: tải bộ nhớ, ghi bộ nhớ, số nguyên, số thực. Nếu một vi lệnh đang đợi cache miss, các vi lệnh độc lập phía sau có thể chạy trước.

Đổi tên thanh ghi. Các thanh ghi logic như `eax` chỉ là tên bên ngoài. Bên trong CPU có nhiều thanh ghi vật lý hơn; RAT ánh xạ thanh ghi logic sang thanh ghi vật lý. Hai đoạn mã độc lập cùng dùng `eax` có thể được đổi thành hai thanh ghi vật lý khác nhau, giúp chúng chạy ngoài thứ tự.

Quy trình chung là: đơn vị nạp trước đọc lệnh, giải mã và đổi tên thanh ghi, đưa vào reorder buffer để sắp lại, gửi tới đơn vị thực thi, giữ kết quả, rồi retirement unit trả kết quả theo đúng thứ tự chương trình. Nếu bước nạp trước sai vì dự đoán nhánh sai, phần sau phải hủy hết, cho thấy nhánh nguy hiểm thế nào.

5.3.1 Giảm phụ thuộc giữa câu lệnh

Ở C, ta không kiểm soát vi lệnh, nhưng có một nguyên tắc vẫn dùng được: giảm phụ thuộc giữa các câu lệnh. Ví dụ:

```
double sum=0;
for(int i=0;i<N;i++)sum+=a[i];
```

với $N = 10\,000\,000$ tốn 88 532 892 chu kỳ. Đổi thành bốn luồng cộng:

```
double s0=0,s1=0,s2=0,s3=0;
for(int i=0;i<N;i+=4){
    s0+=a[i];s1+=a[i+1];
    s2+=a[i+2];s3+=a[i+3];
}
double sum=(s0+s1)+(s2+s3);
```

thời gian giảm còn 43 307 199 chu kỳ. Chuỗi phụ thuộc trên `sum` đã được tách thành bốn chuỗi độc lập, giúp CPU chạy song song tốt hơn.

Một nguyên tắc phụ: trên máy 32 bit, nếu không thiếu bộ nhớ, nên dùng kiểu nguyên 32 bit. Các thanh ghi 16 bit như `ax` chỉ là phần của thanh ghi 32 bit, khó đổi tên hơn và mã lệnh thường dài hơn.

6 Kỹ thuật bit

Mọi dữ liệu trong máy tính đều được lưu nhị phân. Các phép logic bit như `and`, `or`, `not`, `xor`, dịch bit và quét bit gần với cổng logic phần cứng nhất, thường chạy trong một chu kỳ. Kết hợp khéo các phép này có thể thay thao tác chậm hơn.

6.1 Nén bit

Giả sử `a` là biến 32 bit chứa 32 cờ boolean:

Đọc bit k	<code>a>>k&1</code>
Đọc bit k rồi đảo	<code>~a>>k&1</code>
Xóa bit k	<code>a&=~(1<<k)</code>
Bật bit k	<code>a =1<<k</code>
Đảo bit k	<code>a^=1<<k</code>
Đảo đoạn $k_1..k_2$	<code>a^=((1<<(k2-k1+1))-1)<<k2</code>
Đúng một bit bật	<code>!(x&(x-1))&&x</code>
Có hai bit bật kề nhau	<code>x>>1&x</code>
Có ba bit bật kề nhau	<code>x>>1&x>>2&x</code>

6.2 Thống kê trong một gói bit

Đếm từng bit sẽ mất lợi thế của nén. Có thể dùng bảng nhỏ, chẳng hạn 2^{16} , hoặc xử lý bằng phép bit theo kiểu chia đôi.

Đếm parity của số bit bất:

```
x^=x>>1;x^=x>>2;
x^=x>>4;x^=x>>8;
x^=x>>16;
```

Sau đó bit i của kết quả là parity của đoạn từ i tới bit cao nhất. Parity của đoạn $k_1..k_2$ là:

```
(x>>k1^x>>(k2+1))&1
```

Đếm số bit bật:

```
int count(unsigned int x)
{
    x=(x&0x55555555)+(x>>1&0x55555555);
    x=(x&0x33333333)+(x>>2&0x33333333);
    x=(x&0x0F0F0F0F)+(x>>4&0x0F0F0F0F);
    x=(x&0x00FF00FF)+(x>>8&0x00FF00FF);
    x=(x&0x0000FFFF)+(x>>16&0x0000FFFF);
    return x;
}
```

Đối số nên là unsigned int. Với kiểu có dấu, dịch phải có thể là dịch số học (sar) và mở rộng kiểu có thể điền bit dấu, rất dễ gây lỗi khó tìm.

Đảo thứ tự bit:

```
unsigned int rev(unsigned int x){
    x=(x&0x55555555)<<1|(x>>1&0x55555555);
    x=(x&0x33333333)<<2|(x>>2&0x33333333);
    x=(x&0x0F0F0F0F)<<4|(x>>4&0x0F0F0F0F);
    x=(x&0x00FF00FF)<<8|(x>>8&0x00FF00FF);
    x=(x&0x0000FFFF)<<16|(x>>16&0x0000FFFF);
    return x;
}
```

6.3 Loại bỏ nhánh bằng C

```
int abs(int x){
    int y=x>>31;
    return (x+y)^y;
}
```

```
int max(int x,int y){
    int m=(x-y)>>31;
    return y&m|x&~m;
}
```

Nếu x đang bằng một trong hai biến a , b , để chuyển sang biến còn lại:

```
x^=a^b
```

Đổi chỗ không dùng biến phụ:

```
void swap(int& x,int& y){x^=y;y^=x;x^=y;};
```

Tính trung bình hai số nguyên không tràn:

```
int ave(int x,int y){return (x&y)+((x^y)>>1);};
```

7 Lưu ý khi dùng mảng nhiều chiều

Bộ nhớ máy tính là tuyến tính, còn chương trình thường dùng cấu trúc nhiều chiều như ma trận hoặc bảng quy hoạch động 3, 4 chiều. Cần ánh xạ chỉ số nhiều chiều xuống địa chỉ một chiều. Với mảng hai chiều, các hàng được đặt liên tiếp; với nhiều chiều hơn, có thể hiểu đệ quy: đặt x mảng con $(d - 1)$ chiều liên tiếp.

7.1 Chỉ phí định địa chỉ

Ví dụ:

```
int a[10][10][10][10][10];
int main(){
    int x=3,y=7,z=4,w=2,l=1;
    a[x][y][z][w][l]=5;
    return 0;
}
```

gcc sinh mã định địa chỉ dạng:

```
movl    -20(%ebp), %ecx
movl    -24(%ebp), %ebx
movl    -28(%ebp), %esi
movl    -32(%ebp), %edx
movl    -36(%ebp), %edi
movl    %edx, %eax
sall    $2, %eax
addl    %edx, %eax
leal    (%eax,%eax), %edx
imull    $10000, %ecx, %eax
addl    %eax, %edx
imull    $1000, %ebx, %eax
addl    %eax, %edx
imull    $100, %esi, %eax
leal    (%edx,%eax), %eax
addl    %edi, %eax
movl    $5, a(,%eax,4)
```

Chỉ để làm $a[x][y][z][w][l]=5$, đã có 17 lệnh, trong đó 3 lệnh `imul`. Vì vậy trong DP, viết:

```
if(a[x-1][yy][z-3]<a[x][y][z])a[x][y][z]=a[x-1][yy][z-3];
```

có thể đắt hơn nhiều so với:

```
int &mn=a[x][y][z],v=a[x-1][yy][z-3];
```

```
if(v<mn)mn=v;
```

Khi truy cập có quy luật, nên dùng con trỏ và offset đã tính sẵn. Ví dụ sau $a[x][y][z]$ thường là $a[x][y][z+1]$, địa chỉ chỉ cần tăng. Nếu chuyển trạng thái luôn từ $a[x-1][y-2]$ sang $a[x][y]$, có thể tính trước offset giữa hai vị trí.

7.2 Hiệu quả tăng thấp của truy cập mảng

Xét chương trình:

```
int a[2048][2048];
int main(){
    int i,x,y;
    for(i=1;i<=100;i++){
        for(x=0;x<2048;x++)
            for(y=0;y<2048;y++)
                a[x][y]=x+y;
    }
    return 0;
}
```

Duyệt theo hàng, thời gian `user time` là 1.760s. Nếu đổi thứ tự vòng x và y , tức duyệt theo cột, thời gian thành 18.757s, chậm hơn hơn 10 lần. Nếu tăng kích thước mảng thành 2049×2049 , dữ liệu nhiều hơn nhưng thời gian chỉ 4.644s.

Duyệt theo hàng nhanh vì đúng thứ tự lưu trong bộ nhớ, thân thiện với cache. Duyệt theo cột phải nhảy xa, dễ mất dữ liệu trong L1. Yếu tố này giải thích mức 1.7s lên 4.6s. Kết quả 18s còn do kích thước 2048 là lũy thừa của 2. Khi bước nhảy địa chỉ là lũy thừa của 2, các bit dùng để chọn nhóm cache lặp lại, gây xung đột cache liên tục; trong trường hợp xấu, mỗi truy cập đều miss.

Kết luận thực dụng: với mảng nhiều chiều lớn, tránh để kích thước của bất kỳ chiều nào là lũy thừa của 2; tốt nhất dùng kích thước lẻ hoặc nói thêm một chút. DP nén trạng thái thường có một chiều 2^k , lúc đó tăng kích thước mảng thêm ít phần tử đôi khi giúp đáng kể.

8 Ví dụ ứng dụng

Trong nhiều bài thi, thuật toán đúng là đủ lấy điểm tối đa. Nhưng khi chưa tìm được thuật toán tối ưu, các tối ưu hằng số có thể giúp qua thêm 1 đến 4 test ở một số bài; cộng lại vẫn đáng kể.

8.1 Số Mersenne

Nguồn: NOIP 2003, bảng phổ cập.

Bài toán: tính k chữ số thập phân cuối của $2^p - 1$, với $p \leq 3\,100\,000$, $k = 500$.

Thuật toán mặc định: lặp p lần, mỗi lần nhân số lớn với 2, độ phức tạp $\mathcal{O}(kp)$. Kết quả qua 7 test. Tối ưu phép nhân số lớn, loại bỏ phép chia và nhánh điều kiện, qua đủ 10 test.

8.2 Vũ điệu lộn lầy

Nguồn: NOI 2005.

Bài toán có lưới $N \times M$, $N, M \leq 200$. Vật thể bắt đầu tại (X, Y) . Có chuỗi hành động dài $T \leq 40000$, gồm các hướng đông, nam, tây, bắc, được chia thành $S \leq 200$ đoạn, mỗi đoạn có hướng giống nhau. Vật thể đi từng bước theo chuỗi, không được đụng chướng ngại và không ra khỏi lưới. Hỏi cần xóa ít nhất bao nhiêu phần tử trong chuỗi để đi hợp lệ.

DP mặc mạc dùng $F[i][j][k]$: đang ở (j, k) , xét từ đoạn hành động thứ i , số phần tử ít nhất cần xóa. Khi chuyển, liệt kê số phần tử t bị xóa trong đoạn i , độ phức tạp $\mathcal{O}(NMT)$. Bản mặc mạc qua 6 test; tối ưu định địa chỉ mảng nhiều chiều qua đủ 10 test.

8.3 Dịch tác phẩm Maya

Nguồn: IOI 2006.

Bài toán: cho hai xâu W độ dài $G \leq 3000$ và S độ dài $N \leq 3000000$, bảng chữ cái $A = 52$. Đếm số lần một hoán vị của W xuất hiện trong S .

Cài đặt mặc mạc duyệt vị trí bắt đầu trong S , kiểm tra tần suất ký tự của đoạn bằng $\mathcal{O}(A)$, tổng $\mathcal{O}(AN)$. Bản này qua 12 test; tối ưu vòng lặp và định địa chỉ mảng bằng hợp ngữ qua đủ 16 test.

9 Tổng kết

Các kỹ thuật trên rời rạc và không có “thuốc vạn năng”. Nhưng có vài nguyên tắc đáng giữ.

Tối ưu quá sớm là nguồn gốc của nhiều rắc rối. Câu nói nổi tiếng nhờ Donald Knuth rất thực dụng trong thi đấu. Tối ưu có giá: mã khó đọc, khó debug, khó bảo trì, và có thể khiến ta không còn kiểm soát được chương trình vốn có thể viết đúng. Ngoài ra, tối ưu tốn thời gian. Khi viết bản đầu, đặc biệt trong cuộc thi, không nên thêm tối ưu làm tăng độ khó tư duy hoặc cài đặt. Hãy có chương trình đúng trước, rồi tìm bottleneck. Nếu một hàm chỉ chiếm 0.01 giây, tối ưu nó 100 lần cũng không ảnh hưởng đáng kể. Thường 99% thời gian nằm trong 1% mã, hoặc có bài 70% thời gian nằm ở I/O. Hãy tối ưu đúng chỗ; với phần không phải bottleneck, cài đặt rõ ràng là đủ.

Tối ưu không có điểm kết thúc. Sau khi tìm được bottleneck, đừng vội nói đoạn mã không thể nhanh hơn. Điều chỉnh logic ở ngôn ngữ bậc cao, phối hợp với trình biên dịch, hoặc phối hợp với CPU ở tầng hợp ngữ đều có không gian đào sâu. Nhưng tối ưu thật sự đòi hỏi kiến thức và kinh nghiệm về CPU, hợp ngữ, trình biên dịch. Nó có quy luật nhưng cũng đầy ngoại lệ. Cách duy nhất để biết tối ưu có hiệu quả là tự cài và đo được cải thiện.

Keep it simple and stupid. Tối ưu tăng thấp thường làm mọi thứ phức tạp hơn, trái tinh thần KISS. Nhưng khi một tối ưu trở nên quá rối, có thể ta đang đi sai hướng; nên dừng lại và tìm cách đơn giản hơn. Nếu buộc phải viết mã xấu để đạt hiệu năng cực hạn, hãy giới hạn nó vào đúng vài đoạn nóng. Phần còn lại của chương trình vẫn nên giữ đơn giản, rõ ràng và dễ kiểm soát.

Bài viết chỉ chạm tới bề mặt của nhiều nguyên tắc và kỹ thuật. Hy vọng nó gợi mở cho người đọc thấy trong lập trình còn có một vùng thú vị như vậy, và có thể khai thác thêm trong thực chiến.

10 Tài liệu tham khảo

1. *Journey of Games: My Programming Reflections*.
2. *Professional Assembly Language*.
3. *Assembly Language for Intel-Based Computers*, Fifth Edition.
4. *Optimizing Software in C++: An Optimization Guide for Windows, Linux and Mac Platforms*.
5. *Michael Abrash's Graphics Programming Black Book Special Edition*.
6. *Coding ASM: Intel Instruction Set Codes and Cycles*.
7. *Intel 64 and IA-32 Architectures Software Developer's Manual*.
8. *Optimizing Subroutines in Assembly Language: An Optimization Guide for x86 Platforms*.

11 Lời cảm ơn

Tác giả cảm ơn thầy Zhang Junliang vì sự hướng dẫn tận tâm; cảm ơn đàn anh Feng Guodong (bluemary) vì nhiều năm giúp đỡ vô tư; cảm ơn giáo sư Zuo Jie của Đại học Tứ Xuyên vì các ý kiến quý báu cho bài viết; cảm ơn mọi đồng đội và đối thủ đã giúp tác giả trên con đường thi đấu; và cảm ơn cha mẹ cùng giáo viên chủ nhiệm vì sự ủng hộ và bao dung.